

EXPERIMENT 1

AIM:

To design and develop a simple personal portfolio website using basic HTML, CSS, and JavaScript that displays personal details, skills, projects, and contact information

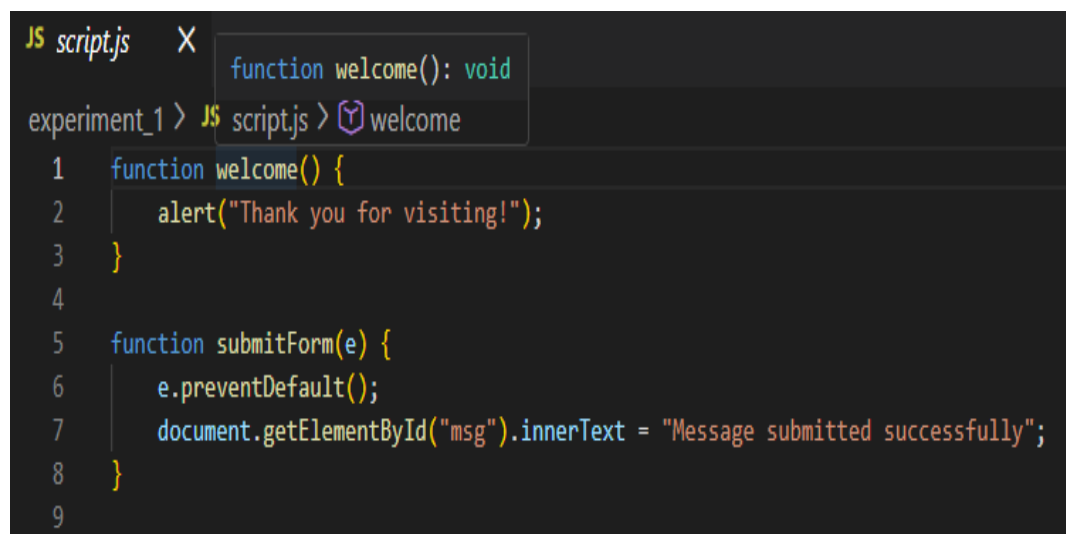
TASKS:

- . Create a project folder named personal portfolio.
- . Create index.html, style.css, and script.js files.
- . Design sections for About, Skills, Projects, and Contact.
- . Apply basic styling using CSS.

TECH STACK USED:

HTML, CSS, JavaScript, VS Code (Editor), Web Browser (Chrome)

CODE:

A screenshot of a Visual Studio Code editor window. The top bar shows a file named 'script.js' with a close button. Below the top bar, a search bar contains 'experiment_1 > JS script.js > welcome'. The main editor area displays the following JavaScript code:

```
1 function welcome() {  
2     alert("Thank you for visiting!");  
3 }  
4  
5 function submitForm(e) {  
6     e.preventDefault();  
7     document.getElementById("msg").innerText = "Message submitted successfully";  
8 }  
9
```

```

index.html X
experiment_1 > index.html > html > body > section#skills > h2
1  <!DOCTYPE html>
2  <html>
3  <head>
4    <title>My Portfolio</title>
5    <link rel="stylesheet" href="style.css">
6  </head>
7  <body>
8
9    <header>
10     <h1>Prince Tiwary </h1>
11     <p>Computer Science Student</p>
12
13     <nav>
14       <a href="#about">About</a>
15       <a href="#skills">Skills</a>
16       <a href="#projects">Projects</a>
17       <a href="#contact">Contact</a>
18     </nav>
19   </header>
20
21   <section>
22     <h2>Welcome</h2>
23     <button onclick="welcome()">Click Here</button>
24   </section>
25
26   <section id="about">
27     <h2>About Me</h2>
28     <p>I am a B.Tech CSE student learning web development.</p>
29   </section>
30
31   <section id="skills">
32     <h2>Skills</h2>
33     <ul>
34       <li>HTML</li>
35       <li>CSS</li>
36       <li>JavaScript</li>
37     </ul>
38   </section>
39
40   <section id="projects">
41     <h2>Projects</h2>
42

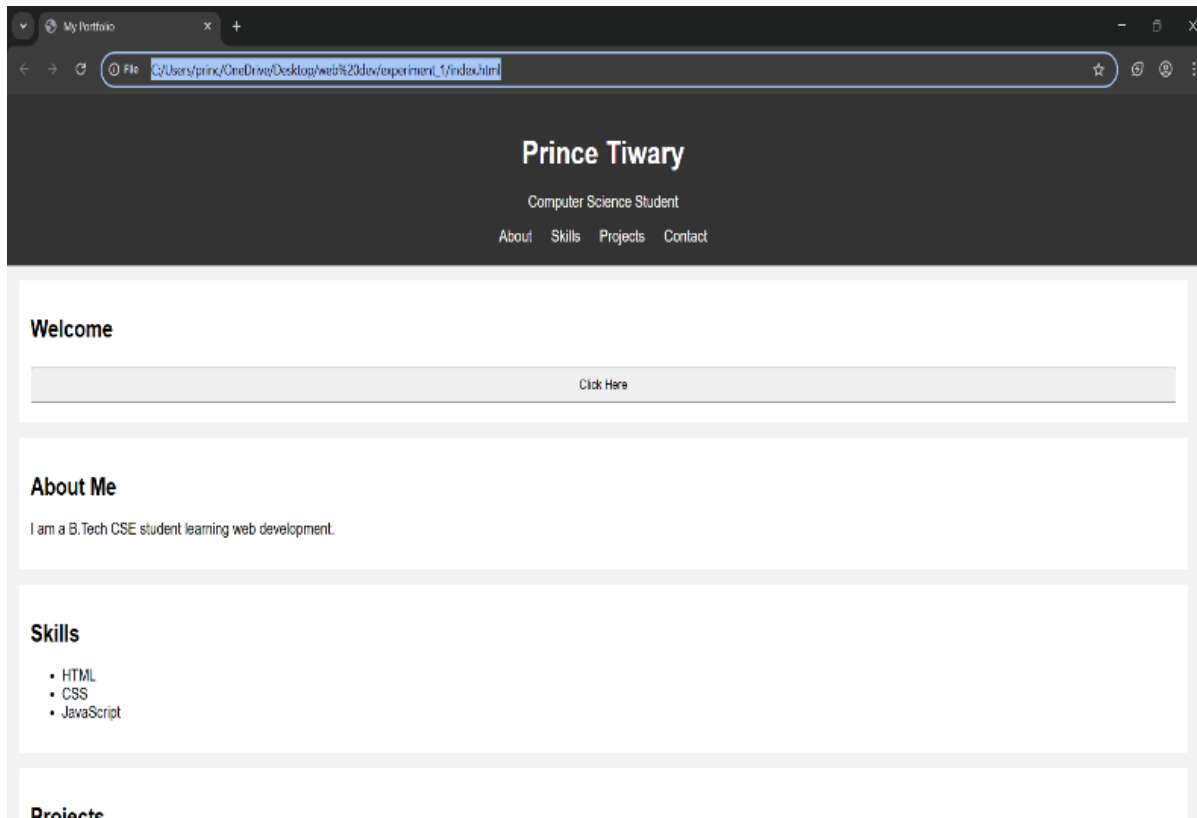
```

```

# style.css X
experiment_1 > # style.css > body
1  body {
2    font-family: Arial;
3    background: #f2f2f2;
4    margin: 0;
5  }
6
7  header {
8    background: #333;
9    color: white;
10   text-align: center;
11   padding: 20px;
12 }
13
14 nav a {
15   color: white;
16   margin: 10px;
17   text-decoration: none;
18 }
19
20 section {
21   background: white;
22   margin: 15px;
23   padding: 15px;
24 }
25
26 div {
27   border: 1px solid #ccc;
28   padding: 10px;
29   margin-top: 10px;
30 }
31
32 input, textarea, button {
33   margin: 5px 0;
34   padding: 8px;
35   width: 100%;
36 }
37
38 footer {
39   background: #333;
40   color: white;
41   text-align: center;
42   padding: 10px;
43 }
44

```

OUTPUT:



EXPERIMENT 2

AIM:

To design and develop a Company Landing Page using HTML and CSS that includes a hero section, about section, services, testimonials, and a contact form.

TASKS

- . Create a project folder named **company_landing_page**.
- . Create **index.html** and **style.css** files inside the folder.
- . Add header comments in HTML (project title, name, date).
- . Apply CSS styling for layout, spacing, colors, fonts, and alignment using box model properties.

TECK STACK USED

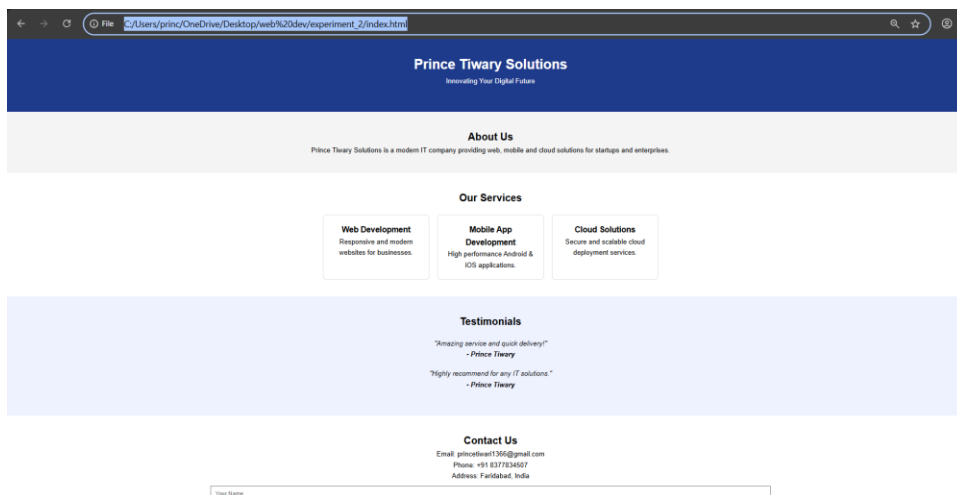
HTML,CSS

CODE

```
index.html X
experiment_2 > index.html > ...
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>PRINCE TIWARY SOLUTIONS</title>
6   <link rel="stylesheet" href="style.css">
7 </head>
8
9 <body>
10
11 <header class="hero">
12   <h1>Prince Tiwary Solutions</h1>
13   <p>Innovating Your Digital Future</p>
14 </header>
15
16 <section id="about">
17   <h2>About Us</h2>
18   <p>
19     Prince Tiwary Solutions is a modern IT company providing web,
20     mobile and cloud solutions for startups and enterprises.
21   </p>
22 </section>
23
24 <section id="services">
25   <h2>Our Services</h2>
26
27   <div class="service-container">
28     <div class="card">
29       <h3>Web Development</h3>
30       <p>Responsive and modern websites for businesses.</p>
31     </div>
32
33     <div class="card">
34       <h3>Mobile App Development</h3>
35       <p>High performance Android & iOS applications.</p>
36     </div>
37
38     <div class="card">
39       <h3>Cloud Solutions</h3>
40       <p>Secure and scalable cloud deployment services.</p>
41     </div>
42   </div>
43 </section>
```

```
# style.css X
experiment_2 > # style.css >  *
1  * {
2    margin: 0;
3    padding: 0;
4    box-sizing: border-box;
5  }
6
7  body {
8    font-family: Arial, sans-serif;
9    line-height: 1.6;
10 }
11
12 .hero {
13   background: #1e3a8a;
14   color: white;
15   text-align: center;
16   padding: 60px 20px;
17 }
18
19 section {
20   padding: 40px;
21   text-align: center;
22 }
23
24 #about {
25   background: #f4f4f4;
26 }
27
28 .service-container {
29   display: flex;
30   justify-content: center;
31   gap: 20px;
32   margin-top: 20px;
33 }
34
35 .card {
36   background: white;
37   padding: 20px;
38   width: 250px;
39   border: 1px solid #ddd;
40   border-radius: 8px;
41 }
42
43 #testimonials {
44   background: #eef2ff;
```

OUTPUT



EXPERIMENT 3

AIM:

To design and develop an **Interactive Quiz Web Application** using HTML, CSS and JavaScript that displays questions, tracks score, shows results and allows restarting the quiz.

TASKS:

- . Create a project folder named **interactive_quiz**.
- . Capture user answers and move between questions.
- . Track and update quiz score dynamically.
- . Display final result and feedback message.
- . Add **Restart Quiz** feature to reuse the app.

TECH STACK USED:

Html, CSS, JavaScript

CODE:

```
index.html X
experiment_3 > index.html > ...
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>Interactive Quiz</title>
6   <link rel="stylesheet" href="style.css">
7 </head>
8
9 <body>
10
11 <div class="container">
12   <h1>General Knowledge Quiz</h1>
13   <p>Test your basic knowledge with this quick quiz.</p>
14
15   <div id="quiz-box">
16     <h2 id="question">Question here</h2>
17     <div id="options"></div>
18     <button id="nextBtn">Next</button>
19   </div>
20
21   <div id="result-box" class="hide">
22     <h2 id="score"></h2>
23     <p id="message"></p>
24     <button onclick="restartQuiz()">Restart Quiz</button>
25   </div>
26 </div>
27
28 <script src="script.js"></script>
29 </body>
30 </html>
31
```

```
# style.css X
experiment_3 > # style.css > body
1 body {
2   font-family: Arial;
3   background: #eef2ff;
4   display: flex;
5   justify-content: center;
6   align-items: center;
7   height: 100vh;
8 }
9
10 .container {
11   background: white;
12   padding: 30px;
13   width: 400px;
14   text-align: center;
15   border-radius: 10px;
16 }
17
18 #options button {
19   display: block;
20   width: 100%;
21   margin: 10px 0;
22   padding: 10px;
23   cursor: pointer;
24 }
25
26 #nextBtn {
27   margin-top: 10px;
28   padding: 10px 20px;
29 }
30
31 .hide {
32   display: none;
33 }
34
```



```

JS script.js x
experiment_3 > JS script.js > ...
1  const questions = [
2      {
3          question: "Capital of India?",
4          options: ["Mumbai", "Delhi", "Chennai", "Kolkata"],
5          answer: "Delhi"
6      },
7      {
8          question: "2 + 2 = ?",
9          options: ["3", "4", "5", "6"],
10         answer: "4"
11     },
12     {
13         question: "HTML stands for?",
14         options: ["Hyper Trainer Markup Language", "Hyper Text Markup Language", "High Text Machine Language", "None"],
15         answer: "Hyper Text Markup Language"
16     }
17 ];
18
19 let currentQ = 0;
20 let score = 0;
21
22 const questionEl = document.getElementById("question");
23 const optionsEl = document.getElementById("options");
24 const nextBtn = document.getElementById("nextBtn");
25
26 function loadQuestion() {
27     optionsEl.innerHTML = "";
28     let q = questions[currentQ];
29     questionEl.textContent = q.question;
30
31     q.options.forEach(option => {
32         let btn = document.createElement("button");
33         btn.textContent = option;
34         btn.onclick = () => selectAnswer(option);
35         optionsEl.appendChild(btn);
36     });
37 }
38
39 function selectAnswer(option) {
40     if (option === questions[currentQ].answer) score++;
41     currentQ++;
42
43     if (currentQ < questions.length) {
44         loadQuestion();

```

OUTPUT:

General Knowledge Quiz

Test your basic knowledge with this quick quiz.

Capital of India?

Mumbai

Delhi

Chennai

Kolkata

Next

EXPERIMENT 4

AIM:

To design and develop a Dynamic To-Do List Web Application that allows users to add, edit, delete, and mark tasks as completed using HTML, CSS, and JavaScript.

TASKS:

- . Created project folder dynamic _to do _list
- . Designed To-Do interface using HTML
- . Implemented dynamic task addition using JavaScript

TECH STACK USED:

HTML,CSS ,JAVASCRIPT

CODE:

```

<> index.html X
experiment_4 > <> index.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <title>Dynamic To Do List</title>
6      <link rel="stylesheet" href="style.css">
7  </head>
8  <body>
9
10 <div class="container">
11     <h1>Dynamic To-Do List</h1>
12     <p class="desc">Add, edit, delete and complete your daily tasks.</p>
13
14     <div class="input-section">
15         <input type="text" id="taskInput" placeholder="Enter a task">
16         <button onclick="addTask()">Add Task</button>
17     </div>
18
19     <ul id="taskList"></ul>
20
21     <div class="counter">
22         Completed: <span id="completed">0</span> |
23         Pending: <span id="pending">0</span>
24     </div>
25 </div>
26
27 <script src="script.js"></script>
28 </body>
29 </html>
30

```

```

# style.css X
experiment_4 > # style.css > body
1  body{
2      font-family: Arial, sans-serif;
3      background: #f4f6f9;
4      display: flex;
5      justify-content: center;
6      align-items: center;
7      height: 100vh;
8  }
9
10 .container{
11     background: white;
12     padding: 25px;
13     width: 400px;
14     border-radius: 10px;
15     box-shadow: 0 0 10px rgba(0,0,0,0.1);
16 }
17
18 h1{
19     text-align: center;
20 }
21
22 .desc{
23     text-align: center;
24     color: gray;
25     margin-bottom: 20px;
26 }
27
28 .input-section{
29     display: flex;
30     gap: 10px;
31 }
32
33 input{
34     flex: 1;
35     padding: 10px;
36     border: 1px solid #ccc;
37     border-radius: 5px;
38 }
39
40 button{
41     padding: 10px 15px;
42     border: none;
43     background: #007bff;
44     color: white;
45     border-radius: 5px;
46 }

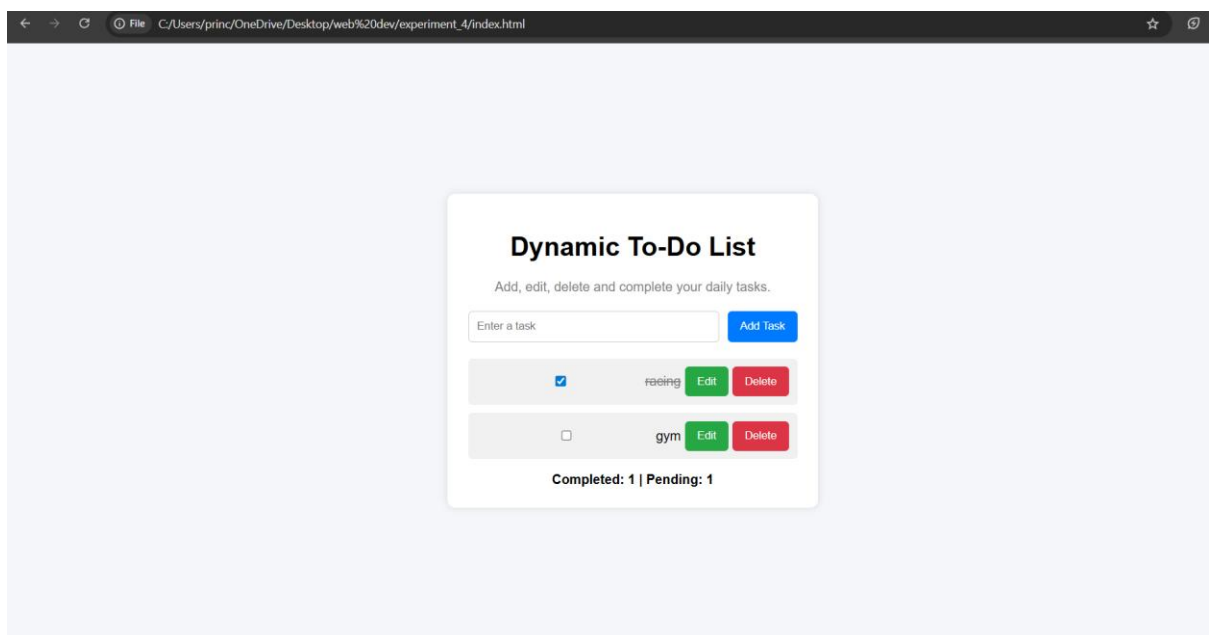
```

```

JS script.js X
experiment_4 > JS script.js > ...
1  let taskList = document.getElementById("taskList");
2  let completedCount = document.getElementById("completed");
3  let pendingCount = document.getElementById("pending");
4
5  function updateCounter(){
6      let tasks = document.querySelectorAll("li");
7      let completed = document.querySelectorAll(".completed");
8      completedCount.textContent = completed.length;
9      pendingCount.textContent = tasks.length - completed.length;
10 }
11
12 function addTask(){
13     let input = document.getElementById("taskInput");
14     let taskText = input.value.trim();
15
16     if(taskText === "") return;
17
18     let li = document.createElement("li");
19
20     let checkbox = document.createElement("input");
21     checkbox.type = "checkbox";
22     checkbox.onclick = function(){
23         li.classList.toggle("completed");
24         updateCounter();
25     };
26
27     let span = document.createElement("span");
28     span.textContent = taskText;
29
30     let btnDiv = document.createElement("div");
31     btnDiv.className = "task-btns";
32
33     let editBtn = document.createElement("button");
34     editBtn.textContent = "Edit";
35     editBtn.onclick = function(){
36         let newTask = prompt("Edit task:", span.textContent);
37         if(newTask !== null && newTask.trim() !== ""){
38             span.textContent = newTask;
39         }
40     };
41
42     let deleteBtn = document.createElement("button");
43     deleteBtn.textContent = "Delete";
44     deleteBtn.className = "delete";

```

OUTPUT:



EXPERIMENT 5

AIM:

To design and develop a Simple Blog Web Application using the Flask framework that performs CRUD operations (Create, Read, Update, Delete) without using a database, and demonstrates the use of templates, routing, and basic styling in web development.

TASK :

Project Setup & Environment

Flask App Initialization

Create Operation

Read Operation

Tech Stack Used:

PYTHON, FLASK, HTML,CSS

CODE:

```
EXPLORER
SIMPLE_BLOG
  static
  style.css
  templates
    base.html
    create.html
    edit.html
    index.html
  app.py

app.py
1
2 from flask import Flask, render_template, request, redirect, url_for
3
4 app = Flask(__name__)
5
6 # Temporary storage (No database used)
7 posts = []
8
9 # HOME - Display all posts
10 @app.route('/')
11 def index():
12     return render_template('index.html', posts=posts)
13
14 # CREATE - Add new post
15 @app.route('/create', methods=['GET', 'POST'])
16 def create():
17     if request.method == 'POST':
18         title = request.form['title']
19         content = request.form['content']
20
21         posts.append({
22             'id': len(posts),
```

```
app.py
style.css
base.html
create.html
edit.html
index.html X

templates > index.html > ...
1 {% extends 'base.html' %}
2
3 {% block content %}
4
5 <h2>📰 Latest Blog Posts</h2>
6
7 {% if posts %}
8     {% for post in posts %}
9         <div class="post">
10             <h3>{{ post.title }}</h3>
11             <p>{{ post.content }}</p>
12
13             <div class="actions">
14                 <a href="/edit/{{ post.id }}" class="edit">Edit</a>
15                 <a href="/delete/{{ post.id }}" class="delete">Delete</a>
16             </div>
17         </div>
18     {% endfor %}
19 {% else %}
20     <p class="empty">No posts yet. Create your first post 🚀</p>
21 {% endif %}
```

```
# style.css create.html x ... create.html edit.html x
templates > create.html > ...
1 {% extends 'base.html' %}
2
3 {% block content %}
4
5 <h2>Create New Post ✨</h2>
6
7 <form method="POST" class="form-card">
8   <input type="text" name="title" placeholder="Title" value="">
9
10  <textarea name="content" placeholder="Content" required>
11
12  <button type="submit">Publish Post</button>
13 </form>
14
15 {% endblock %}
```

```
templates > edit.html > ...
1 {% extends 'base.html' %}
2
3 {% block content %}
4
5 <h2>Edit Post ✏️</h2>
6
7 <form method="POST" class="form-card">
8   <input type="text" name="title" value="">
9
10  <textarea name="content" required>{{ content }}
11
12  <button type="submit">Update Post</button>
13 </form>
14
15 {% endblock %}
```

```
# style.css create.html x ... create.html edit.html x
templates > create.html > ...
1 {% extends 'base.html' %}
2
3 {% block content %}
4
5 <h2>Create New Post ✨</h2>
6
7 <form method="POST" class="form-card">
8   <input type="text" name="title" placeholder="Title" value="">
9
10  <textarea name="content" placeholder="Content" required>
11
12  <button type="submit">Publish Post</button>
13 </form>
14
15 {% endblock %}
```

```
templates > edit.html > ...
1 {% extends 'base.html' %}
2
3 {% block content %}
4
5 <h2>Edit Post ✏️</h2>
6
7 <form method="POST" class="form-card">
8   <input type="text" name="title" value="">
9
10  <textarea name="content" required>{{ content }}
11
12  <button type="submit">Update Post</button>
13 </form>
14
15 {% endblock %}
```

OUTPUT:

